CODING5S ECOSYSTEM | PEDAGOGY QUICK READ

Pillar 1 — The 5-Stage Learning Architecture

Curriculum & Authoring Layer of the Coding5s Framework — Creator Kit, Curriculum Generation, and Controlled Cognitive Friction

EXECUTIVE SUMMARY

Pillar 1 is the curriculum-and-authoring layer of Coding5s. It delivers a spreadsheet-native **Creator Kit** that transforms any technical topic into a structured five-stage practice sequence. The goal is not frictionless learning — it is **Controlled Cognitive Friction**: AI accelerates the surrounding scaffolding while the learner retains ownership of reasoning, execution, debugging, and decision-making.

Practice → Debug → Complete → Refactor → Extend

01 The 5-Stage Learning Architecture

Each topic is transformed into five related forms of practice. Exact prompt structure, mentor behavior, and difficulty vary by language, library, domain, and learner level — the stages define the learning **progression**, not a rigid template.

STAGE 01 🟥 Practice

Build initial familiarity

OBJECTIVE & AI PERSONA

- Read small reference examples, reproduce and execute them, observe results, make small changes.
- AI generates examples/explanations, but never replaces direct hands-on interaction.

KEY STUDENT TAKEAWAYS

- Connect syntax with observable behavior.
- Move from passive observer to active operator of the tool or pattern.

STAGE 02 🟨 Debug

Develop corrective competence

OBJECTIVE & AI PERSONA

- Learner receives broken, incomplete, or misleading material tied to Stage 1 content (syntax, logic, config, data faults).
- AI asks questions, interprets errors, and hints — it does not perform the full correction.

KEY STUDENT TAKEAWAYS

- Diagnostic reasoning is preserved and exercised, not outsourced.
- Learner builds fluency in reading errors and failure signatures.

STAGE 03 🟩 Complete

Recognition → construction

OBJECTIVE & AI PERSONA

- Part of a working artifact is intentionally missing (function, query, config, workflow step).
- AI supplies enough surrounding context for the learner to reason about the gap.

KEY STUDENT TAKEAWAYS

- Independent construction from partial context.
- First real transition toward authorship of a working component.

STAGE 04 🟦 Refactor

Improve what already works

OBJECTIVE & AI PERSONA

- Evaluate the Stage 3 result against readability, maintainability, performance, robustness, security.
- Rules stay **ecosystem-specific** — Python OOP, Elixir functional, SQL, and networking are never forced through one design lens.

KEY STUDENT TAKEAWAYS

- Idiomatic, domain-appropriate design judgment.
- Trade-off reasoning between correctness and quality.

STAGE 05 🟪 **Extend** *Transfer to new requirements*

OBJECTIVE & AI PERSONA

- Learner adds a new capability, constraint, integration, or scaling requirement to the existing artifact.
- AI introduces the new requirement; learner owns the redesign.

KEY STUDENT TAKEAWAYS

- Understand → modify under new requirements → explain/defend decisions.
- Proof of transfer beyond reproducing the original lesson.

02 Controlled Cognitive Friction

Coding5s does not prohibit AI-generated code — it controls when and how AI assistance enters the learning task.

Depending on the stage, AI may provide reference examples, broken code, partial implementations, scaffolding, error explanations, diagnostic questions, analogies, review feedback, or new requirements.

> **AI can support the work, but it should not silently replace the cognitive task the learner is supposed to practice.**

03 The Creator Kit

The Creator Kit is the spreadsheet-based authoring environment used to construct Coding5s curricula — combining topics, stage-specific prompt components, learner-level variables, ecosystem rules, output-language controls, and reusable formulas into structured prompts usable with capable LLMs. It is intentionally spreadsheet-native so an educator can modify the learning architecture without building a custom application.

Curriculum + Technical Rules + Stage Rules + Config Variables + Optional Context → **Creator Kit** → **Generated Learning Prompts** → **Student Practice**

Language & Context Adaptation

The generated course can target any selected human output language. For languages where ordinary model performance needs stronger grounding, an optional **Language Seed Context** can be incorporated. Technical subject and human teaching language are treated as separate configuration dimensions.

Supporting Practice Tools

Complementary generators — e.g. the **Synthetic Practice Dataset Generator** — create small reproducible CSV, JSON, XLSX, log, or NumPy artifacts engineered around the lesson topic, so file-based exercises use real data instead of imagined data. These extend, but never replace, the five-stage architecture.

04 Reference Creator Kits

Reference implementations shipped in-repo — not the boundary of Pillar 1. The same architecture adapts to additional languages, libraries, and domains.

ECOSYSTEM	AVAILABLE CREATOR KITS
Dart	Coding5s Dart Fundamentals Creator Kit v0.2.xlsx
Elixir	Coding5s Elixir Fundamentals Creator Kit v0.2.xlsx · Coding5s Elixir OTP Creator Kit v0.2.xlsx
Python	Core & Scripting v0.1 · HTTPX v0.1 · Numpy v0.1 · Pandas v0.1 · Requests v0.1 (all Creator Kit v0.1.xlsx)

05 Cross-Model Reference Executions

Historical execution samples document the same stage instructions run across independent LLMs — treated as reference samples, not formal benchmarks. Model behavior can change over time and shared links may become unavailable.

CURRICULUM	STAGES COVERED	MODELS REFERENCED
💧 Elixir Fundamentals	Stage 1 – 5	Deepseek, Gemini, Grok
🐍 Python Core & Scripting	Stage 1 – 5	Deepseek, Grok, ChatGPT

06 Getting Started

1. **Select a reference Creator Kit** or start from the master template.
2. **Define the target** curriculum, technology, learner level, and output language.
3. **Configure** the technical and stage-specific rules.
4. **Generate and inspect** the resulting prompts.
5. **Test representative topics** before producing the full Student Kit.
6. **Iterate** when model behavior exposes ambiguity or excessive assistance.

Full generation workflow: see **CURRICULUM_GENERATOR.md**.

07 Relationship to the Other Pillars

Pillar 1

What does the learner practice? How is the curriculum structured?

Pillar 2

How should the AI mentor behave during that practice?

Pillar 3

What useful context should persist as the learner progresses?

The three pillars work together, but Pillar 1 remains independently useful as the curriculum and practice architecture of Coding5s.

TECHNICAL FOOTER

KEY ARCHITECTURAL PILLARS

- 5-Stage progression: Practice → Debug → Complete → Refactor → Extend
- Spreadsheet-native Creator Kit authoring layer
- Ecosystem-specific refactor & extend rules
- Separated technical-subject / output-language configuration

SYSTEM GUARANTEES

- Controlled Cognitive Friction preserved at every stage
- AI assists scaffolding, never silently replaces the target skill
- No platform build required to author or adapt curricula
- Model-agnostic prompt generation (cross-LLM compatible)

REFERENCES

- CURRICULUM_GENERATOR.md — full generation workflow
- Reference Creator Kits — Dart, Elixir, Python (v0.1–v0.2)
- Synthetic Practice Dataset Generator
- Pillar 2 (AI Mentor Behavior) · Pillar 3 (Persistent Context)